

UNIVERSIDADE FEDERAL DE SANTA CATARINA

Josué Elias Fajardo

Gabriel Menezes

Ararangua

2026

Projeto 1

Proposta

1. Estude e diga que hardware o VHDL ao lado
Implementa

2. Gere um testbench e simule este hardware

3. Remova um dos sinais da lista de sensitividade do process e mostre o que isto muda no comportamento do hardware, via nova simulação com o mesmo testbench. Guarde ambas versões do arquivo para entregar, salvando as formas de onda e marcando onde estas diferem

4. Desenhe um diagrama de esquemáticos do circuito (com portas lógicas e/ou FFs e/ou multiplexadores e/ou etc.)

```
library IEEE;
use IEEE.std_logic_1164.all;
entity prim_proc is
  port( in1, in2, in3, in4 : in std_logic;
        ctrl : in std_logic_vector (1 downto 0);
        sai : out std_logic);
end prim_proc;
architecture prim_proc of prim_proc is
begin
  process (in1, in2, in3, in4, ctrl)
  begin
    case ctrl is
      when "00"   => sai <= in1;
      when "01"   => sai <= in2;
      when "10"   => sai <= in3;
      when "11"   => sai <= in4;
      when others => null;
    end case;
   end process;
end prim_proc;
```

Solução

O hardware descreve um multiplexador de 4 inputs para 1 output. O mesmo testbench foi usado para cada caso

```
entity prim_proc is
    port( in1, in2, in3, in4 : in std_logic;
    ctrl : in std_logic_vector(1 downto 0);
    sai : out std_logic);
end prim_proc;

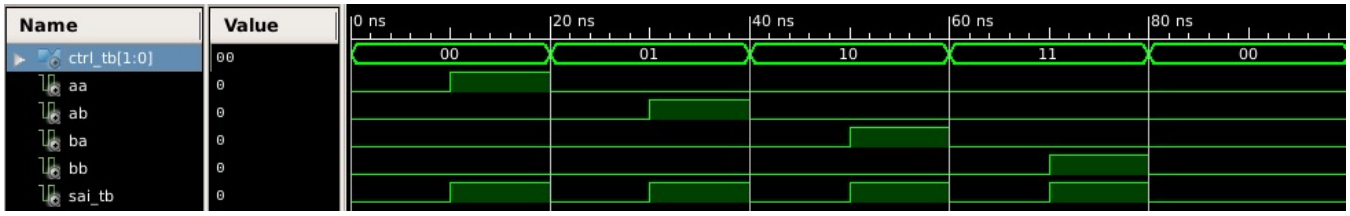
architecture prim_proc of prim_proc is
begin
    process (in1, in2, in4, ctrl)
    begin
        case ctrl is
            when "00" => sai <= in1;
            when "00" => sai <= in1;
            when "01" => sai <= in2;
            when "10" => sai <= in3;
            when "11" => sai <= in4;
            when others => null;
        end case;
    end process;
end prim_proc;
```

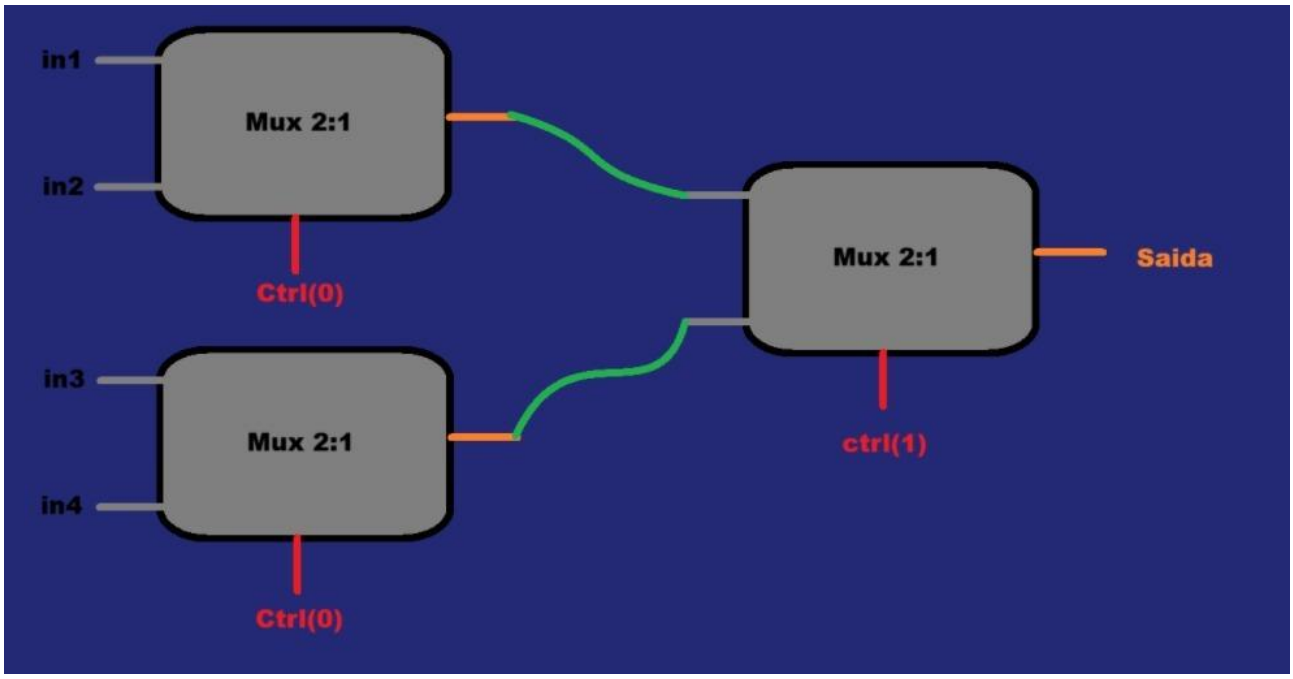
```
entity prim_proc_tb is
end prim_proc_tb;

architecture TB_architecture of prim_proc_tb is
    signal aa, ab, ba, bb : std_logic;
    signal ctrl_tb : std_logic_vector(1 downto 0);
    signal sai_tb : std_logic;
begin
UUT: entity work.prim_proc
        port map(in1 => aa, in2 => ab, in3 => ba, in4 => bb, ctrl => ctrl_tb, sai => sai_tb);

        ctrl_tb <= "00", "01" after 20 ns, "10" after 40 ns, "11" after 60 ns, "00" after 80 ns;
        aa <= '0', '1' after 10 ns, '0' after 20 ns;
        ab <= '0', '1' after 30 ns, '0' after 40 ns;
        ba <= '0', '1' after 50 ns, '0' after 60 ns;
        bb <= '0', '1' after 70 ns, '0' after 80 ns;

end TB_architecture;
```


Name
Value
ctrl_tb[1:0]
aa
ab
ba
bb
sai_tb
00
0
0
0
0
0
0 ns
20 ns
40 ns
60 ns
80 ns
00
01
10
11
00


in1
in2
Mux 2:1
Ctrl(0)
in3
in4
Mux 2:1
Ctrl(0)
Mux 2:1
ctrl(1)
Saida

Projeto 2

Proposta

1. Estude e diga que hardware o VHDL ao lado implementa (ele foi gerado a partir do ChatGPT!)

2. Estude o uso do comando VHDL for-generate e depois gere uma versão equivalente deste hardware que empregue este comando (i.e., gere uma arquitetura alternativa para a mesma entidade) e que não empregue o comando process

3. Gere um testbench e simule ambas as versões (arquiteturas) deste hardware

4. Desenhe um diagrama de esquemáticos do circuito (no nível de portas lógicas) para N=4

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity n_bit_d is
    generic (N: integer := 2);
    port (input: in std_logic_vector (N-1 downto 0);
          output: out std_logic_vector ((2**N)-1 downto 0));
end entity n_bit_d;

architecture n_bit_d of n_bit_d is
begin
    process (input)
        variable int_input : integer;
    begin
        output <= (others => '0');
        int_input := to_integer(unsigned(input));
        output(int_input) <= '1';
    end process;

end architecture n_bit_d;
```

Solução

O código gerado corresponde a um codificador one-hot, que “seleciona” um bit equivalente da saída. Uma entrada de N bits possui uma saída de 2^N bits.

Ex. 011 = 00001000 ; 000 = 00000001 ;

O código original usa process para gerar o hardware, enquanto a solução usa for-generate para gerar o mesmo hardware.

```
entity n_bit_one_hot is
    generic (
        n : natural := 3
    );
    port (
        input : in std_logic_vector(n-1 downto 0) := (others => '0');
        output : out std_logic_vector((2**n)-1 downto 0)
    );

end n_bit_one_hot;

architecture Behavioral of n_bit_one_hot is
begin

    gen: for i in 0 to ((2**n)-1) generate
        output(i) <= '1' when unsigned(input) = i else '0';
    end generate gen;

end Behavioral;
```

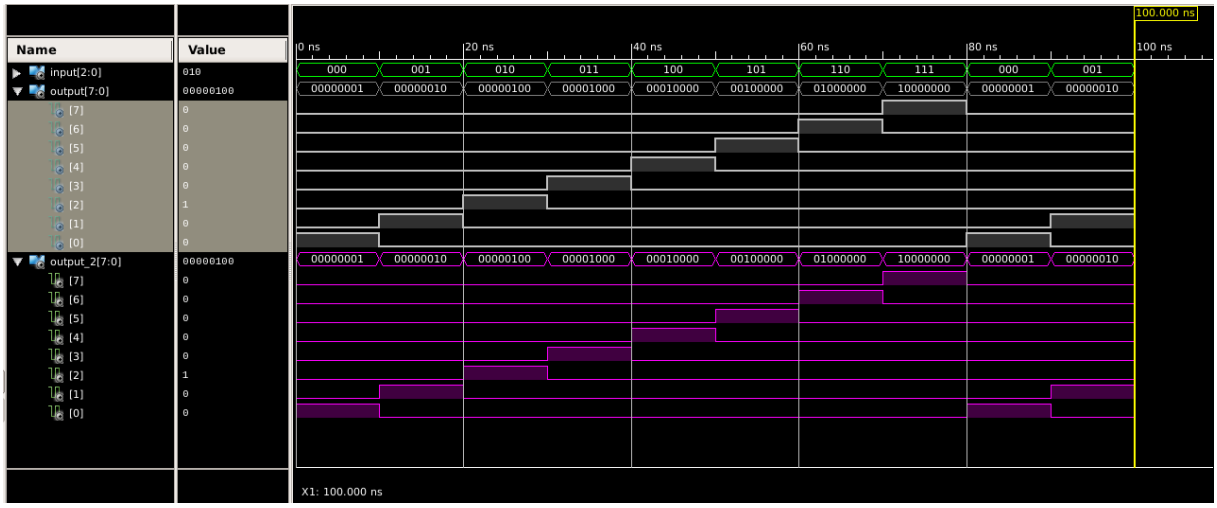
```
ARCHITECTURE behavior OF one_hot_tb IS
    constant num: integer := 3;
    signal input: std_logic_vector (num-1 downto 0) := "000";
    signal output, output_2: std_logic_vector ((2**num)-1 downto 0);
BEGIN
    UUT: entity work.n_bit_d
    generic map (N => num)
    port map (input => input,
              output => output);

    UUT_2: entity work.n_bit_one_hot
    generic map (n => num)
    port map (input => input,
              output => output_2);



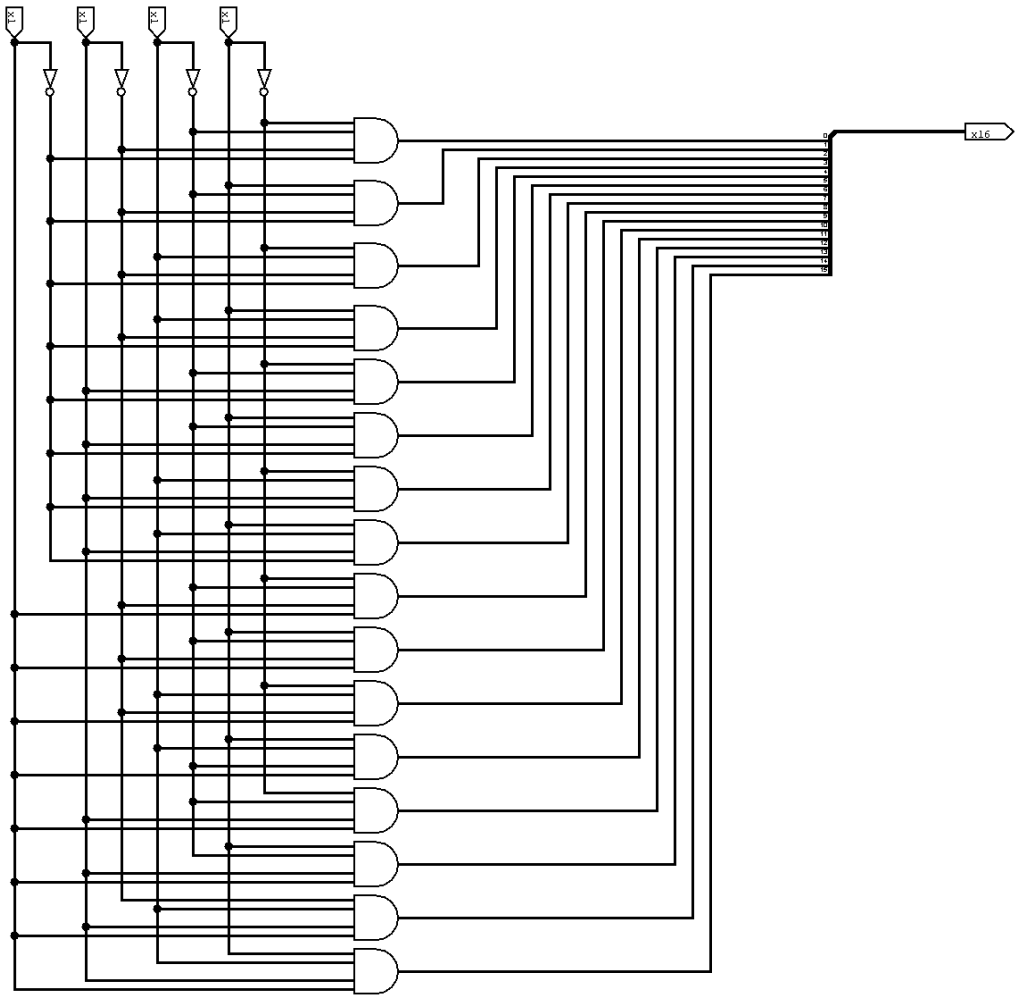
    process (input)
    begin
        if (input = "111") then
            input <= "000" after 10 ns;
        else
            input <= std_logic_vector(unsigned(input) + "001") after 10 ns;
        end if;
    end process;

END;
```



Em ciano, o código original. Em roxo, o código sem process.

O codificador pode ser montado usando os mintermos da tabela verdade, assim o circuito, em portas lógicas, fica assim:



Projeto 3

Proposta

1. Estude e diga que hardware o VHDL ao lado implementa (ele foi gerado a partir da AI Gemini!)

2. O texto ao lado possui 29 linhas. Descreva para que serve cada uma delas no relatório do projeto (mesmo as linhas em branco!!!), justificando a existência de cada uma das linhas

3. Gere um testbench e simule este hardware

4. Desenhe um diagrama de esquemáticos do circuito (com portas lógicas e flip-flops) para este circuito. Recomenda-se usar o gerador automático esquemáticos do ISE para ajudar, aprenda a usar e use esta ferramenta

5. Produza uma nova versão do mesmo hardware que use reset assíncrono e mostre que ele funciona (c/ testbench)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity m17_counter is
   port (clk, reset : in  STD_LOGIC;
         count : out STD_LOGIC_VECTOR (4 downto 0));
end m17_counter;

architecture m17_counter of m17_counter is
signal int_count : unsigned(4 downto 0) := (others => '0');
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                int_count <= (others => '0');
            else
                if int_count = 16 then
                    int_count <= (others => '0');
                else
                    int_count <= int_count + 1;
                end if;
            end if;
        end if;
    end process;

    count <= std_logic_vector(int_count);
end m17_counter;
```

30

Solução

Como o nome da entidade indica, é um contador de modulo 17, contando de 0 a 16, com reset sincrono.

O mesmo testbench foi usado para cada caso

```
architecture m17_counter of m17_counter is
signal int_count : unsigned(4 downto 0) := (others => '0');
begin
    process(clk, reset)
    begin
         if reset = '1' then
              int_count <= (others => '0');
         elsif rising_edge(clk) then
              if int_count = 16 then
                    int_count <= (others => '0');
              else
                    int_count <= int_count + 1;
              end if;
         end if;
    end process;
    count <= std_logic_vector(int_count);
end m17_counter;
```

```
entity m17_counter_tb is
end m17_counter_tb;

architecture TB_architecture of m17_counter_tb is
        signal clktb, resettb : STD_LOGIC;
        signal counter : STD_LOGIC_VECTOR (4 downto 0);

begin
    UUT : entity work.m17_counter
        port map(clk => clktb , reset => resettb, count => counter);

        clktb <= '0', not clktb after 5 ns;
        resettb <= '0', '1' after 100 ns, '0' after 110 ns;

end TB_architecture;
```

Project 3 desc

como o nome da entidade indica, é um contador de modulo 17, contando de 0 a 16, com reset sincrono.

O mesmo testbench foi usado para cada caso

descrição de linha:

```
library IEEE;                   declaração de bilbioteca
use IEEE.STD_LOGIC_1164.ALL;    declaração de qual funcção da biblioteca esta sendo usada: tudo em Standard logic 1164
use IEEE.NUMERIC_STD.ALL;       declaração de qual funcção da biblioteca esta sendo usada: tudo em Numeric standard

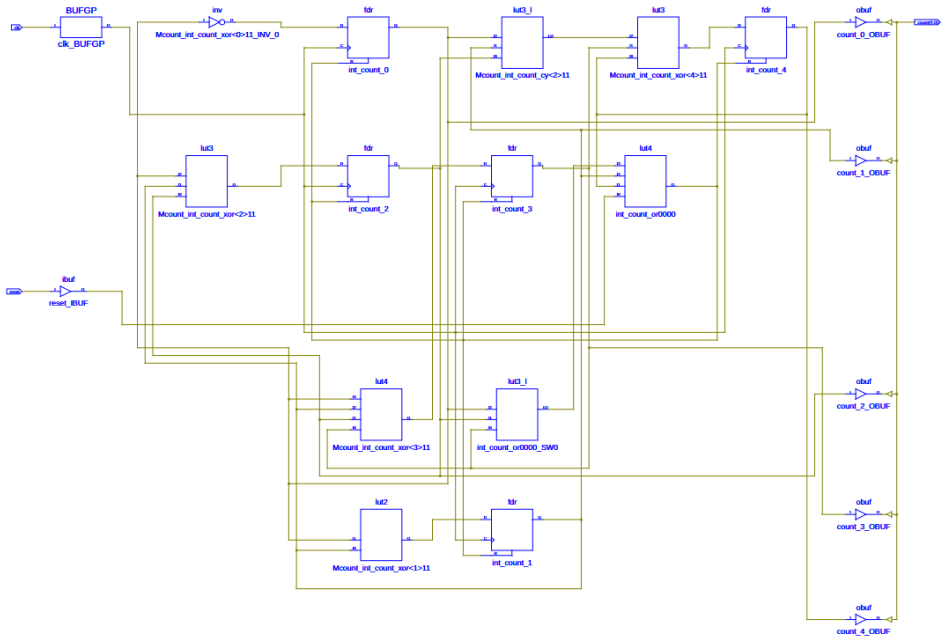
entity m17_counter is                           Declaração do nome
port (clk, reset : in STD_LOGIC;                Declaração de inputs: Clock e Reset em 1 bit Standard logic
count : out STD_LOGIC_VECTOR (4 downto 0));     Declaração de output counter em standard logic vector 5 bits
end m17_counter;                                fim da declaração

architecture m17_counter of m17_counter is                  Definição de funcionalidade.
signal int_count : unsigned(4 downto 0) := (others => '0');     Definição dos "fios" internos: Integer counter como unsigned 5 bit vector, iniciado em '00000'

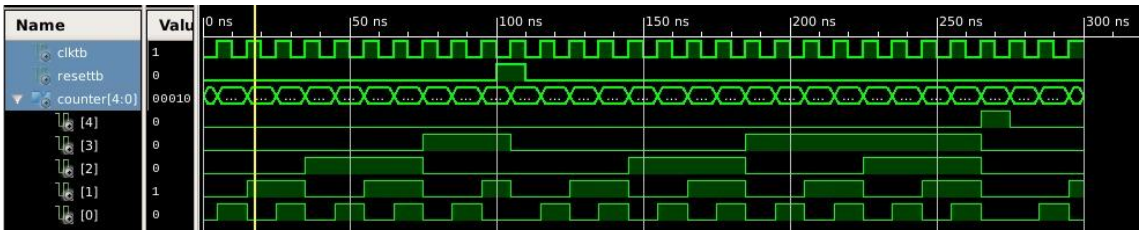
begin                           Declaração de inicio do comportamento.

        process(clk)                                                    Declaração de um processo sequencial com sensitividade a porta Clock
        begin                                                           Declaração de inicio do comportamento.
                if rising_edge(clk) then                                condicional "Se" monitorando a subida de borda de onda clock
                        if reset = '1' then                             condicional "se" ao "se" anterior, se reset for 1
                                int_count <= (others => '0');           zera todos os bits do integer counter
                        else                                            condição alternativa a  (If reset = 1)
                                if int_count = 16 then                  condicional se monitorando quando integer counter = 16 ou '10000'
                                        int_count <= (others => '0');   zera todos os bits do integer counter
                                else                                    condicional alternativo para (if int_count =16)
                                        int_count <= int_count + 1;     adiciona o valor numerico 1 no integer counter
                                end if;                                 finalizaçao do condicional "se" (int_count =16)
                        end if;                                         finalização do condicional "se" (reset =1)
                end if;                                                 finalização do condicional "se" (rising_edge(clk))
        end process;                                                    finalização do processo sequencial com sensitividade a porta clock.

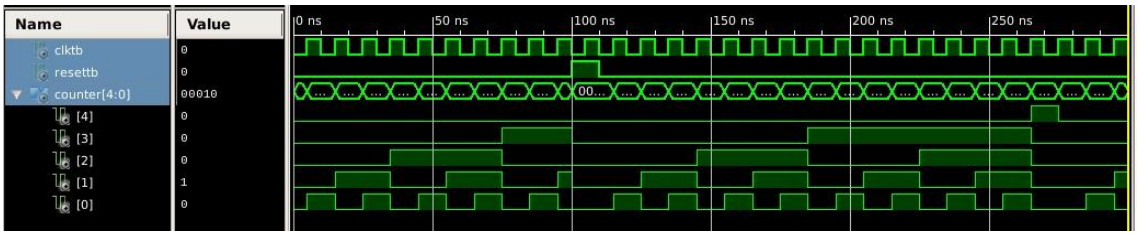
        count <= std_logic_vector(int_count);           saida counter recebe o vetor de integer counter
end m17_counter                                         finalização da funcionalide.
```



Síncrono



Assíncrono



Projeto 4

Proposta

Considere o circuito TX mostrado no próximo slide, o qual implementa uma transmissão serial de dados. A “linha” de dados (saída do circuito) está by default (i.e. por omissão) em ‘1’, indicando que não há transmissão de dados, e que a linha está então em “repouso”.

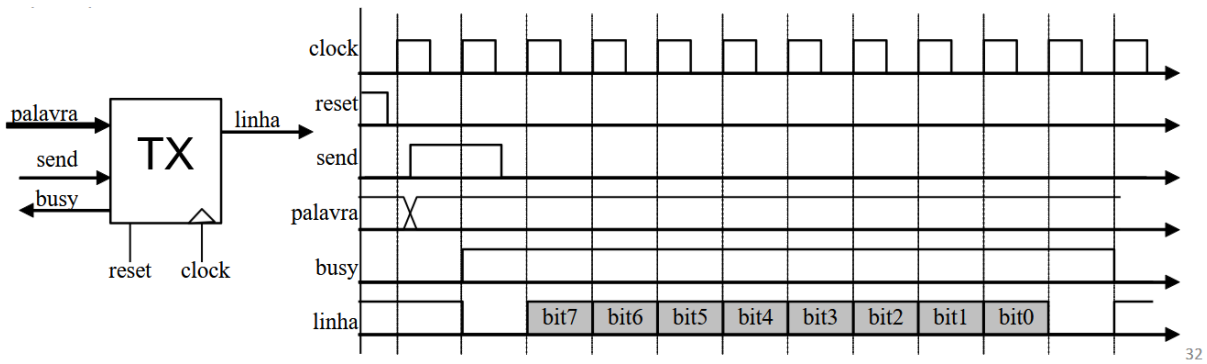
O protocolo de transmissão é o seguinte:

1. O mundo externo à direita de TX (testbench) coloca um byte em “palavra”, e sobe o sinal “send”, indicando ao módulo TX que há dado a ser enviado para a “linha”

2. Na primeira borda de subida do clock após a subida de “send” o módulo TX sobe o sinal de “busy”, avisando que o mundo externo não deve retirar o dado enquanto ele processa o dado atual. Concorrentemente a esta ação a linha sai do repouso, indo a 0 por um ciclo (no que se denomina start bit)

3. Nos próximos 8 ciclos de clock, o dado no sinal “palavra” é transferido, bit a bit, para a “linha” serial, indo do bit mais significativo (bit 7) até o bit menos significativo (bit 0)

4. No décimo ciclo de clock após a detecção de “send”, “linha” vai para 0 (o que se denomina stop bit) e o sinal “busy” desce no final deste último ciclo. Depois, “linha” vai para 1



Solução

O transmissor foi feito usando 3 estados:

1. Idle: o transmissor espera um sinal de “send”.
2. Sending: o transmissor emite os bits armazenados no buffer.
3. Finishing: o transmissor terminou de enviar os bits, e fica em ‘0’ por um ciclo de clock.

```
entity transmissor is
    port( clock, reset, send : in std_logic;
            busy, linha : out std_logic;
            palavra : std_logic_vector(7 downto 0));
end transmissor;

architecture Behavioral of transmissor is
    signal buf : std_logic_vector(7 downto 0);
    type STATE is (idle, sending, finishing);
    signal current_state : STATE := idle;
begin

    busy <= '0' when (current_state = idle or current_state = finishing) else '1' when current_state = sending;

    process(clock, reset)
        variable current_bit : integer := 8; -- Conta de 8 a 0, enviando o bit armazenado no buffer
    begin
        if clock'event and clock='1' then
            if (current_state = idle) then
                if ((send='1') and (current_state /= sending)) then
                    current_state <= sending;
                    buf <= palavra;
                    linha <= '0';
                else
                    linha <= '1';
                end if;
            elsif (current_state = sending) then
                if ( current_bit = 0) then
                    current_bit := 8;
                    linha <= '0';
                    current_state <= finishing;
                else
                    current_bit := current_bit-1;
                    linha <= buf(current_bit);
                end if;
            elsif (current_state = finishing) then
                current_state <= idle;
                linha <= '1';
            end if;
        end if;

        if reset = '1' then
            current_state <= idle;
            linha <= '1';
            buf <= (others => '0');
            current_bit := 8;
        end if;
    end process;

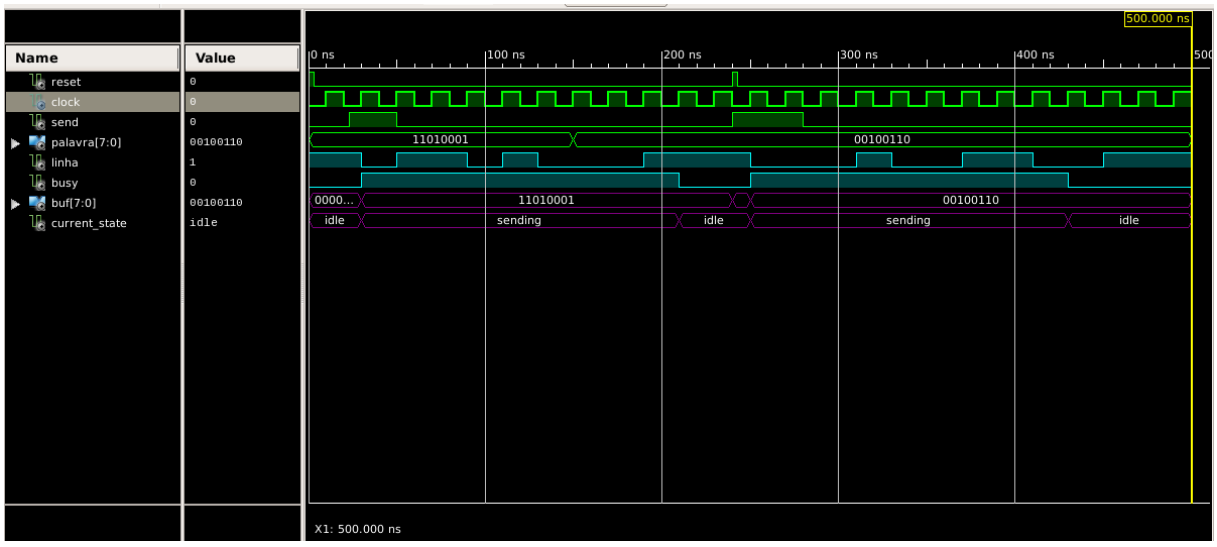
end Behavioral;
```

```
architecture transmissor_tb of transmissor_tb is
    signal busy, linha, reset, send: std_logic;
    signal clock: std_logic:='0';
    signal palavra : std_logic_vector(7 downto 0);
begin

    UUT : entity work.transmissor port map (clock => clock, reset => reset, send => send,
    palavra => palavra, busy => busy, linha => linha);

    process
    begin
        clock <= '1' after 10ns, '0' after 20ns;
        wait for 20ns;
    end process;

    reset <= '1', '0' after 3 ns, '1' after 240ns, '0' after 243 ns;
    send <= '0', '1' after 23 ns, '0' after 50 ns, '1' after 240ns, '0' after 280 ns;
    palavra <= "11010001", "00100110" after 150ns;
end transmissor_tb;
```



Em ciano os outputs, em roxo os elementos internos.